Prefix Sum of Matrix (Or 2D Array)

Difficulty: **Medium** Accuracy: **74.81%** Submissions: **222+** Points: **4** Average Time: **20m**

Given a integer matrix (or 2D array) **a[][]** of dimensions **n * m**. Also, given another 2-D array **query[][]** of dimensions **q * 4**.

For each index 0 < i < query.length, find the sum of all the elements of the rectangular matrix whose top left corner is (query[i][0], query[i][1]) and bottom right corner is (query[i][2], query[i][3]).

Example -

Input:
n = 3, m = 3, q = 2

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

214 / 214

Attempts : Correct / Total

You can see all your attempts in submission tab

Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public static ArrayList<Long> submatrixSum(long[][] a, int n, int m, int[][] query, int q) {
        long[][] pre = new long[n][m];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                pre[i][j] = a[i][j];

                if (i > 0) pre[i][j] += pre[i - 1][j];
                if (j > 0) pre[i][j] += pre[i][j - 1];
                if (i > 0 && j > 0) pre[i][j] -= pre[i - 1][j - 1];
            }
        }

        ArrayList<Long> ans = new ArrayList<>();

        for (int i = 0; i < q; i++) {
            int r1 = query[i][0];
            int c1 = query[i][1];
            int r2 = query[i][2];
            int c2 = query[i][3];

            long sum = pre[r2][c2];

            if (r1 > 0) sum -= pre[r1 - 1][c2];
            if (c1 > 0) sum -= pre[r2][c1 - 1];
            if (r1 > 0 && c1 > 0) sum += pre[r1 - 1][c1 - 1];

            ans.add(sum);
        }

        return ans;
    }
}
```

Custom Input Compile & Run Submit

Footpath Construction

Difficulty: **Medium** Accuracy: **39.47%** Submissions: **218+** Points: **4** Average Time: **19m**

Given a matrix **a** of size **n*m** which represents a **park**, there is some construction work needs to be done. You are also given **q** queries each query contains two numbers **R** and **C**, For every query we need to construct a footpath in the **Rth** row and **Cth** column, there is a **cost** of this construction, after the construction this path will divide the park into **sections**, and the cost of the construction is the **sum** of **minimum** value present in all the sections. You are asked to find this cost for all the queries.

Note: Elements present in queries array are according to 1-based indexing.

Example 1:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1122 / 1122

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int[] createFootpath(int n, int m, int a[][], int q, int queries[][]) {
        int INF = Integer.MAX_VALUE;

        int[][] tl = new int[n][m];
        int[][] tr = new int[n][m];
        int[][] bl = new int[n][m];
        int[][] br = new int[n][m];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                tl[i][j] = a[i][j];
                if (i > 0) tl[i][j] = Math.min(tl[i][j], tl[i - 1][j]);
                if (j > 0) tl[i][j] = Math.min(tl[i][j], tl[i][j - 1]);
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = m - 1; j >= 0; j--) {
                tr[i][j] = a[i][j];
                if (i > 0) tr[i][j] = Math.min(tr[i][j], tr[i - 1][j]);
                if (j < m - 1) tr[i][j] = Math.min(tr[i][j], tr[i][j + 1]);
            }
        }

        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j < m; j++) {
                bl[i][j] = a[i][j];
                if (i < n - 1) bl[i][j] = Math.min(bl[i][j], bl[i + 1][j]);
                if (j > 0) bl[i][j] = Math.min(bl[i][j], bl[i][j - 1]);
            }
        }

        for (int i = n - 1; i >= 0; i--) {
            for (int j = m - 1; j >= 0; j--) {
                br[i][j] = a[i][j];
                if (i < n - 1) br[i][j] = Math.min(br[i][j], br[i + 1][j]);
                if (j < m - 1) br[i][j] = Math.min(br[i][j], br[i][j + 1]);
            }
        }

```

Custom Input Compile & Run Submit

Minimum Steps to Halve Sum

Difficulty: **Medium** Accuracy: **55.59%** Submissions: **14K+** Points: **4**

Given an array **arr[]**, find the **minimum** number of operations required to make the sum of its elements less than or equal to **half** of the original sum. In one operation, you may replace any element with half of its value (with **floating-point** precision).

Examples:

Input: arr[] = [8, 6, 2]
Output: 3
Explanation: Initial sum = (8 + 6 + 2) = 16, half = 8
Halve 8 → arr[] = [4, 6, 2], sum = 12 (still 12 > 8)
Halve 6 → arr[] = [4, 3, 2], sum = 9 (still 9 > 8)

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.Collections;
import java.util.PriorityQueue;

class Solution {
    public int minOperations(int[] arr) {
        double currentSum = 0;

        PriorityQueue<Double> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        for (int num : arr) {
            currentSum += num;
            maxHeap.add((double) num);
        }

        double target = currentSum / 2.0;
        int operations = 0;

        while (currentSum > target) {
            double largest = maxHeap.poll();
            double halved = largest / 2.0;

            currentSum -= halved;
            maxHeap.add(halved);

            operations++;
        }

        return operations;
    }
}
```

Custom Input Compile & Run Submit

Minimum Number of Workers

Difficulty: **Medium** Accuracy: **55.15%** Submissions: **14K+** Points: **4**

You are given an array **arr[]** of size **n** , where arr[i] denotes the range of working hours a person at position **i** can cover.

- If **arr[i]** ≠ **-1**, the person at index i can work and cover the time interval **[i - arr[i], i + arr[i]]**.
- If **arr[i] = -1**, the person is unavailable and cannot cover any time.

The task is to find the **minimum** number of people required to cover the entire working day from **0 to n - 1**. If it is not possible to fully cover the day, return **-1**.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1122 / 1122	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minMen(int arr[]) {
        int n = arr.length;
        int[] maxRange = new int[n];

        for (int i = 0; i < n; i++) {
            maxRange[i] = -1;
        }

        for (int i = 0; i < n; i++) {
            if (arr[i] != -1) {
                int left = Math.max(0, i - arr[i]);
                int right = Math.min(n - 1, i + arr[i]);
                maxRange[left] = Math.max(maxRange[left], right);
            }
        }

        int count = 0;
        int currentEnd = -1;
        int nextEnd = -1;

        for (int i = 0; i < n; i++) {
            nextEnd = Math.max(nextEnd, maxRange[i]);

            if (i > currentEnd) {
                count++;
                currentEnd = nextEnd;

                if (currentEnd < i) {
                    return -1;
                }
            }
        }

        return count;
    }
}
```

Custom Input Compile & Run Submit

Previous Smaller Element

Difficulty: **Medium** Accuracy: **58.12%** Submissions: **91K+** Points: **4**

You are given an integer array **arr[ ]**. For every element in the array, your task is to determine its **Previous Smaller Element (PSE)**.

The Previous Smaller Element (PSE) of an element **x** is the first element that appears to the left of **x** in the array and is strictly smaller than **x**.

Note: If no such element exists, assign **-1** as the PSE for that position.

Examples:

Input: arr[] = [1, 6, 2]
Output: [-1, 1, 1]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public static ArrayList<Integer> prevSmaller(int[] arr) {
        ArrayList<Integer> result = new ArrayList<>();
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < arr.length; i++) {
            while (!stack.isEmpty() && stack.peek() >= arr[i]) {
                stack.pop();
            }

            if (stack.isEmpty()) {
                result.add(-1);
            } else {
                result.add(stack.peek());
            }

            stack.push(arr[i]);
        }

        return result;
    }
}
```

Custom Input Compile & Run Submit

Previous Greater Element

Difficulty: **Medium** Accuracy: **65.92%** Submissions: **8K+** Points: **4**

You are given an integer array **arr[]**. For every element in the array, determine its **Previous Greater Element (PGE)**.

The Previous Greater Element (PGE) of an element **x** is the first element that appears to the left of **x** in the array and is strictly greater than **x**.

Note: If no such element exists, assign **-1** as the PGE for that position.

Examples:

Input: arr[] = [10, 4, 2, 20, 40, 12, 30]
Output: [-1, 10, 4, 2, 20, 40, 40]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public ArrayList<Integer> preGreaterEle(int[] arr) {
        ArrayList<Integer> result = new ArrayList<>();
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < arr.length; i++) {
            while (!stack.isEmpty() && stack.peek() <= arr[i]) {
                stack.pop();
            }

            if (stack.isEmpty()) {
                result.add(-1);
            } else {
                result.add(stack.peek());
            }

            stack.push(arr[i]);
        }

        return result;
    }
}
```

Custom Input Compile & Run Submit

Subarrays with First Element Minimum

Difficulty: **Medium** Accuracy: **56.75%** Submissions: **18K+** Points: **4**

You are given an integer array **arr[ ]**. Your task is to **count** the number of subarrays where the first element is the **minimum element** of that subarray.

Note: A subarray is valid if its first element is not greater than any other element in that subarray.

Examples:

Input: arr[] = [1, 2, 1]
Output: 5
Explanation:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1120 / 1120

Attempts : Correct / Total

You can see all your attempts in submission tab

Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.Stack;

class Solution {
    public int countSubarrays(int[] arr) {
        int n = arr.length;
        int[] nse = new int[n];
        Stack<Integer> st = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && arr[st.peek()] >= arr[i]) {
                st.pop();
            }

            if (st.isEmpty()) {
                nse[i] = n;
            } else {
                nse[i] = st.peek();
            }
            st.push(i);
        }

        int totalSubarrays = 0;
        for (int i = 0; i < n; i++) {
            totalSubarrays += (nse[i] - i);
        }

        return totalSubarrays;
    }
}
```

Custom Input Compile & Run Submit

Find 132 Pattern in Array

Difficulty: **Medium** Accuracy: **80.66%** Submissions: **1K+** Points: **4**

You are given an array **arr[].** The task is to determine whether the array contains a **132 pattern**, i.e., three indices i, j and k such that i < j < k , **arr[i] < arr[j] > arr[k]** and **arr[i] < arr[k]**.
Return **true** if such a triplet exists, otherwise return **false**.

Examples:

Input: arr[] = [4, 7, 11, 5, 13, 2]
Output: true
Explanation: Triplet [4, 7, 5] satisfies the condition since 4 < 7, 5 < 7 and 4 < 5.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.Stack;

class Solution {
    public boolean has132Pattern(int[] arr) {
        int n = arr.length;
        if (n < 3) return false;

        int third = Integer.MIN_VALUE;
        Stack<Integer> stack = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {
            if (arr[i] < third) {
                return true;
            }

            while (!stack.isEmpty() && arr[i] > stack.peek()) {
                third = stack.pop();
            }

            stack.push(arr[i]);
        }

        return false;
    }
}
```

Custom Input Compile & Run Submit

Maximum People Visible in a Line

Difficulty: **Medium** Accuracy: **50.11%** Submissions: **17K+** Points: **4**

You are given an array **arr[ ]**, where arr[i] represents the height of the ith person standing in a line.
A person **i** can see another person **j** if:

- height[j] < height[i],
- There is no person **k** standing between them such that height[k] ≥ height[i].

Each person can see in both directions (front and back).
Your task is to find the **maximum number of people** that any person can see (including themselves).

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 2** Accuracy : 50%

Points Scored Time Taken

Java (21) Start Timer

```java
import java.util.Stack;

class Solution {
    public int maxPeople(int[] arr) {
        int n = arr.length;
        int[] leftCount = new int[n];
        int[] rightCount = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
                stack.pop();
            }
            leftCount[i] = stack.isEmpty() ? i : i - stack.peek() - 1;
            stack.push(i);
        }

        stack.clear();
        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
                stack.pop();
            }
            rightCount[i] = stack.isEmpty() ? (n - 1) - i : stack.peek() - i - 1;
            stack.push(i);
        }

        int maxVisible = 0;
        for (int i = 0; i < n; i++) {
            maxVisible = Math.max(maxVisible, leftCount[i] + rightCount[i] + 1);
        }

        return maxVisible;
    }
}
```

Custom Input Compile & Run Submit

Unique K-Number Sum

Difficulty: **Medium** Accuracy: **66.08%** Submissions: **12K+** Points: **4**

Given two integers **n** and **k**, the task is to find all valid combinations of **k** numbers that adds up to **n** based on the following conditions:

- **Only** numbers from the range [1, 9] used.
- Each number can only be used **at most** once.

Note: You can return the combinations in any order, the driver code will print them in sorted order.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public ArrayList<ArrayList<Integer>> combinationSum(int n, int k) {
        ArrayList<ArrayList<Integer>> result = new ArrayList<>();
        backtrack(result, new ArrayList<>(), k, n, 1);
        return result;
    }

    private void backtrack(ArrayList<ArrayList<Integer>> result, ArrayList<Integer> tempList,
        if (tempList.size() == k && remain == 0) {
            result.add(new ArrayList<>(tempList));
            return;
        }

        if (tempList.size() == k || remain < 0) {
            return;
        }

        for (int i = start; i <= 9; i++) {
            tempList.add(i);
            backtrack(result, tempList, k, remain - i, i + 1);
            tempList.remove(tempList.size() - 1);
        }
    }
}
```

Custom Input Compile & Run Submit

Equal Partition

Difficulty: **Medium** Accuracy: **42.52%** Submissions: **1K+** Points: **4**

Given an array **arr[]** of size **n**, your task is to divide the array in **two** subsets such that the **absolute difference** between the **sum of elements** in the two subsets is equal to **zero** (i.e., both subsets have the same sum).

- If **n** is **even**, both subsets must contain exactly **n/2** elements.
- If **n** is **odd**, one subset must contain **(n-1)/2** elements and the other subset must contain **(n+1)/2** elements.

Note : If multiple answers exist, you may return any of them. The driver code will check and print **true** if your partition is valid, otherwise **false**.
It is **guaranteed** that there will always be **atleast one** valid partition.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
        if (currentSubset.size() == targetSize) {
            if (currentSum == targetSum) {
                ArrayList<Integer> subset2 = new ArrayList<>();
                boolean[] used = new boolean[arr.length];

                List<Integer> temp = new ArrayList<>(currentSubset);
                for(int val : temp) {
                    for(int i=0; i<arr.length; i++) {
                        if(!used[i] && arr[i] == val) {
                            used[i] = true;
                            break;
                        }
                    }
                }

                for(int i=0; i<arr.length; i++) {
                    if(!used[i]) subset2.add(arr[i]);
                }

                result.add(new ArrayList<>(currentSubset));
                result.add(subset2);
                return true;
            }
            return false;
        }

        if (index >= arr.length) return false;

        currentSubset.add(arr[index]);
        if (findPartition(arr, index + 1, currentSum + arr[index], targetSize, targetSum, currentSu
            return true;
        }
        currentSubset.remove(currentSubset.size() - 1);
        if (findPartition(arr, index + 1, currentSum, targetSize, targetSum, currentSubset, result)
            return true;
        }

        return false;
    }
}
```

Custom Input Compile & Run Submit

Maximize the minimum difference between k elements

Difficulty: **Medium** Accuracy: **61.58%** Submissions: **14K+** Points: **4**

Given an array **arr[]** of integers and an integer **k**, select k elements from the array such that the **minimum absolute difference** between any two of the selected elements is **maximized**. Return this **maximum** possible **minimum** difference.

Examples:

Input: arr[] = [2, 6, 2, 5], k = 3
Output: 1
Explanation: 3 elements out of 4 elements are to be selected with a minimum difference as large as possible. Selecting 2, 2, 5 will result in minimum difference as 0. Selecting 2, 5, 6 will result in minimum difference as 6 - 5 = 1.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;

class Solution {
    public int maxMinDiff(int[] arr, int k) {
        Arrays.sort(arr);

        int n = arr.length;
        int low = 1;
        int high = arr[n - 1] - arr[0];
        int result = 0;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (canPlace(arr, k, mid)) {
                result = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }

    private boolean canPlace(int[] arr, int k, int dist) {
        int count = 1;
        int lastPlaced = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] - lastPlaced >= dist) {
                count++;
                lastPlaced = arr[i];
                if (count >= k) return true;
            }
        }
        return false;
    }
}
```

Custom Input Compile & Run Submit

Count elements less than or equal to k in a sorted rotated array

Difficulty: **Medium** Accuracy: **55.56%** Submissions: **13K+** Points: **4** Average Time: **15m**

Given a sorted array **arr[]** containing distinct non negative integers that has been rotated at some unknown pivot, and a value **x**. Your task is to **count** the number of elements in the array that are **less** than or equal to **x**.

Examples:

Input: arr[] = [4, 5, 8, 1, 3], x = 6
Output: 4
Explanation: 1, 3, 4 and 5 are less than 6, so the count of all elements less than 6 is 4.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1113 / 1113

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int countLessEqual(int[] arr, int x) {
        int n = arr.length;
        int pivot = findPivot(arr, 0, n - 1);

        int count = countInSorted(arr, 0, pivot - 1, x) +
                    countInSorted(arr, pivot, n - 1, x);

        return count;
    }

    private int findPivot(int[] arr, int low, int high) {
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] > arr[high]) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }

    private int countInSorted(int[] arr, int low, int high, int x) {
        int start = low;
        int ans = 0;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] <= x) {
                ans = mid - start + 1;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return ans;
    }
}
```

Custom Input Compile & Run Submit

Add numbers in array

Difficulty: **Basic** Accuracy: **87.41%** Submissions: **2K+** Points: **1** Average Time: **2m**

You will be given a vector of integers of size n containing the elements. Your task is to find the sum of all the integers present in the array.As the sum can be large you have to return a value in long long data type.

Example 1:

Input:
n=7
input= {6,2,5,4,5,1,6}

Output : 29

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1 / 1

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public long get_Sum(int n, int[] input) {
        long sum = 0;

        for (int i = 0; i < n; i++) {
            sum += input[i];
        }

        return sum;
    }
}
```

Custom Input Compile & Run Submit